Industr.IA

E-book

IA agêntica:

Arquitetura de Agentes e Padrões Aplicados

Curso base de IA | Unidade M2

Prof.: Arthur Nascimento

SUMÁRIO

1. Arquitetura canônica de um agente 3
2. Agente e assistente conversacional 7
3. Panorama de paradigmas em alto nível 10
4. Model Context Protocol como padrão de integração 14
5. Dois padrões aplicados em PD&I industrial 17
6. Reconhecimento do padrão em uma especificação 20
7. Conclusão 23

Unidade M2

1. Arquitetura canônica de um agente

Esta seção pertence à camada conceitual e tem por objetivo que você descreva os componentes canônicos de um agente e o ciclo que os articula. A meta não é dominar a implementação de cada componente, deferida ao curso de LLMs da Fase 2, mas reconhecer a forma comum a que sistemas chamados de agentes obedecem, qualquer que seja o framework usado para construí-los. Quem reconhece essa forma consegue ler uma solução proposta, nomear suas partes e prever onde estarão seus custos e seus riscos antes de entrar em qualquer detalhe de código.

Um agente, neste curso, é um sistema de software que recebe um objetivo, decide por conta própria quais ações tomar para alcançá-lo, executa essas ações sobre um ambiente por meio de ferramentas, observa o resultado e repete o processo até concluir a tarefa ou atingir uma condição de parada. A palavra que carrega o peso da definição é decide. Não basta que o sistema execute uma sequência de chamadas; é preciso que ele escolha a próxima ação em função do que observou, e que essa escolha não esteja inteiramente fixada de antemão por quem o programou.

Para fixar a intuição, contraste o agente com uma única chamada de modelo. Numa chamada isolada, você fornece uma entrada, o modelo produz uma saída e o processo termina ali; não há ação sobre o mundo nem observação de consequências. Num agente, a mesma capacidade do modelo é colocada dentro de um laço: o sistema decide uma ação, executa-a, lê o que voltou e decide de novo, à luz desse retorno. A diferença não está no modelo, que pode ser o mesmo nos dois casos, mas na estrutura que o envolve. É essa estrutura, e não uma qualidade especial do modelo, que transforma um gerador de texto num sistema que age.

O componente que comanda o laço é o reasoning engine (módulo de raciocínio). É ele que, a cada passo, lê o estado atual da tarefa, formado pelo objetivo, pelo histórico de ações e pelas observações acumuladas, e decide o que fazer em seguida: invocar uma ferramenta com determinados argumentos, registrar um resultado parcial ou encerrar com uma resposta final. Na maioria das arquiteturas atuais, o reasoning engine é um modelo de linguagem grande, mas a função que ele exerce, decidir a próxima ação, é conceitualmente independente da tecnologia que a realiza. Separar a função do substrato tem consequência prática: quando se confunde o motor de decisão com o modelo específico, trocar o modelo parece exigir reescrever o sistema inteiro; quando se mantém a distinção, fica claro o que é decisão de arquitetura e o que é capacidade do modelo, e a substituição de um pelo outro passa a ser uma questão localizada.

GLOSSÁRIO

Reasoning engine

Componente de um agente que decide, a cada passo do laço, qual será a próxima ação. Lê o estado da tarefa e produz uma decisão (chamar uma ferramenta, registrar um resultado ou parar). Em geral é desempenhado por um modelo de linguagem, mas a função de decisão é distinta do modelo que a executa.

Em torno do reasoning engine organizam-se três outros componentes. O primeiro são as ferramentas, comumente chamadas de tools: as funções e APIs que o agente pode invocar para agir sobre o mundo, como consultar um banco de dados, chamar um serviço externo, executar um cálculo determinístico ou escrever um arquivo. Sem tools, o agente apenas raciocina em texto e nada acontece fora dele; são as tools que convertem uma decisão em efeito observável. Convém distinguir tools de leitura, que apenas trazem informação, de tools de escrita, que alteram o estado do mundo, porque as segundas concentram a maior parte do risco. Vale também notar que um número excessivo de tools tende a degradar a decisão: quanto mais opções, mais frequente o erro de seleção, motivo pelo qual conjuntos pequenos e bem delimitados costumam ser preferíveis.

O segundo componente é a memória. Como o agente opera em vários passos, ele precisa reter o que descobriu entre uma iteração e a seguinte. A memória de curto prazo é o contexto imediato da tarefa em andamento, o histórico de ações e observações que cabe na janela do modelo e que é reapresentado a cada decisão. A memória de longo prazo, quando o sistema a exige, é um armazenamento mais duradouro, externo à janela, que sobrevive a uma execução e de onde o agente recupera informação sob demanda. A janela do modelo é finita, e essa finitude impõe um limite concreto: tarefas longas, com muitas observações, esbarram no espaço disponível.

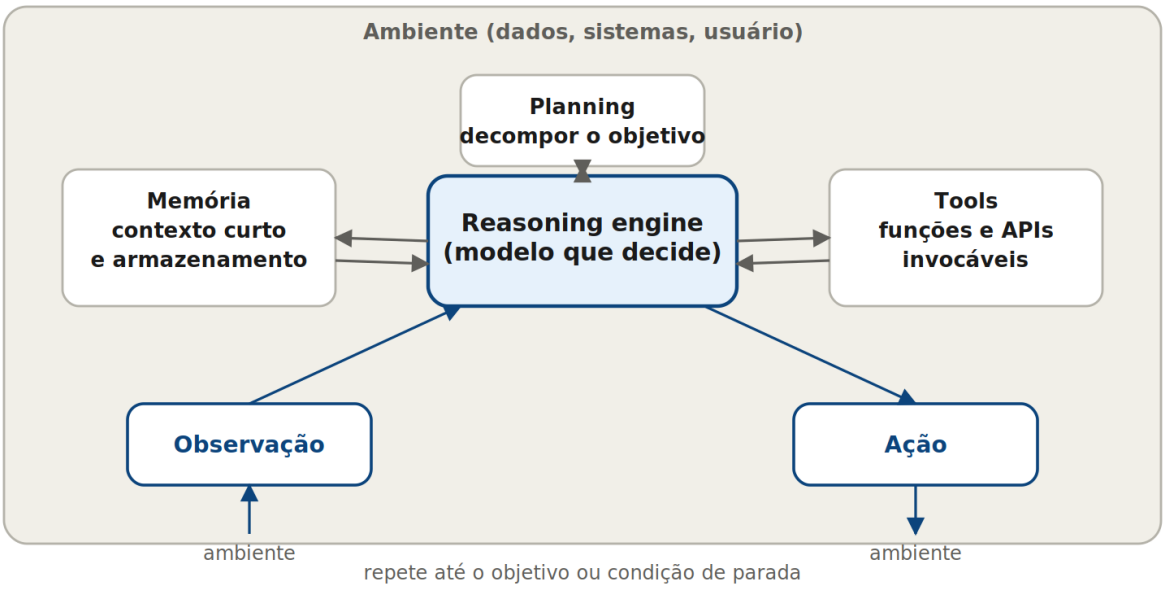
Gerir a memória de curto prazo é, na prática, gerir um recurso escasso. Como tudo o que o agente precisa considerar a cada decisão tem de caber na janela do modelo, e essa janela é finita, o projeto de um agente envolve escolher o que permanece no contexto em sua forma original, o que é resumido para ocupar menos espaço e o que é movido para um armazenamento externo, de onde será recuperado apenas quando necessário. Essas escolhas não são neutras: manter histórico demais encarece cada chamada e pode confundir o modelo com ruído, enquanto descartar demais faz o agente perder o fio da tarefa. Vale notar que o excesso não é inofensivo nem quando tudo cabe na janela, pois informação relevante enterrada num contexto longo tende a ser subutilizada pelo modelo. Reconhecer que a memória é um recurso a ser administrado, e não um

repositório infinito, evita desenhos que funcionam em demonstração e quebram quando a tarefa cresce.

O terceiro componente é o planejamento, ou planning, a capacidade de decompor um objetivo amplo em uma sequência de subtarefas. Em alguns agentes o planejamento é implícito, emergindo passo a passo do próprio raciocínio; em outros é explícito, com o agente elaborando um plano antes de executar. Planejar de forma explícita ajuda em tarefas longas e estruturadas, em que perder o rumo é caro, mas introduz a necessidade de revisar o plano quando as observações divergem do previsto. Os três componentes, tools, memória e planning, não operam isoladamente: é o reasoning engine que os mobiliza, decidindo a cada passo qual tool chamar, o que consultar na memória e quando o plano precisa de ajuste.

Esses componentes se articulam num ciclo que vale a pena nomear, porque reaparece em toda a literatura de agentes: perceber, decidir, agir. O agente percebe o estado atual a partir de uma observação, o reasoning engine decide a próxima ação considerando o objetivo e a memória, e o agente age sobre o ambiente por meio de uma tool. O ambiente devolve uma nova observação, e o ciclo recomeça. Um detalhe frequentemente subestimado é a condição de parada. Sem um critério explícito de quando encerrar, seja por objetivo atingido, por um número máximo de iterações ou por um orçamento de erro esgotado, o laço pode não terminar, e um agente que não para é um problema tanto de custo quanto de confiabilidade. A figura a seguir organiza esses elementos em alto nível.

Figura 1 - Arquitetura canônica de um agente em alto nível.



Fonte: autor.

Figura 1 intitulada "Arquitetura canônica de um agente em alto nível", apresentada como um diagrama em blocos que representa o ciclo perceber, decidir e agir. Todo o fluxo está contido em um grande retângulo de cantos arredondados identificado como "Ambiente (dados, sistemas, usuário)". No centro há um bloco destacado em azul com o texto "Reasoning engine (modelo que decide)", representando o núcleo responsável pela tomada de decisões. Acima dele está um bloco menor chamado "Planning – decompor o objetivo", ligado ao mecanismo central por uma seta para baixo. À esquerda do bloco central encontra-se "Memória – contexto curto e armazenamento", conectado por setas de mão dupla ao mecanismo de decisão. À direita está "Tools – funções e APIs invocáveis", também ligado ao bloco central por setas em ambos os sentidos. Na parte inferior esquerda há o bloco "Observação", que recebe informações do ambiente por uma seta ascendente e envia essas informações ao mecanismo central por uma seta diagonal. Na parte inferior direita está o bloco "Ação", que recebe uma seta diagonal vinda do mecanismo central e envia seus resultados ao ambiente por uma seta apontando para baixo. Na base da figura aparece a indicação de que esse processo se repete até que o objetivo seja alcançado ou uma condição de parada seja atendida, caracterizando um ciclo contínuo de observação, planejamento, decisão e ação. Na parte inferior consta a legenda "Fonte: autor."

Um ponto que merece atenção desde já é a observabilidade do laço. Como o agente decide em vários passos, o registro dessas decisões, o chamado trace, deixa de ser um detalhe de depuração e passa a ser o principal artefato para entender, auditar e corrigir o sistema. O trace guarda, para cada iteração, o raciocínio que levou à ação, a ferramenta invocada, os argumentos passados e a observação retornada. Sem esse registro, um comportamento indesejado vira uma caixa-preta: sabe-se que a resposta final saiu errada, mas não em que passo a decisão se desviou. Projetar um agente, portanto, inclui projetar como o seu trace será capturado e inspecionado, tema que reaparece quando o módulo trata de observabilidade na camada decisória.

É instrutivo acompanhar o ciclo num caso concreto. Um agente que precisa responder qual foi a variação de um indicador entre dois períodos primeiro decide consultar a fonte de dados, executa essa consulta por uma tool, observa os valores retornados, decide então calcular a diferença, executa o cálculo por outra tool, observa o resultado e, julgando-o suficiente, decide encerrar com a resposta. Cada passo foi uma decisão tomada à luz da observação anterior, e o número de passos não estava fixado: outro caso, com dados ausentes, levaria o mesmo agente a consultar uma fonte alternativa antes de responder.

Exemplo aplicado

Numa operação de suporte técnico de uma fabricante de equipamentos industriais, um agente recebe um chamado, consulta a base de manuais por uma tool de busca, verifica o histórico do equipamento em outra tool e só então redige a resposta com o procedimento de manutenção. O reasoning engine decide, a cada passo, se já tem informação suficiente ou se precisa de outra consulta, e o número de consultas varia conforme o chamado.

Uma observação fecha a seção e previne uma confusão comum. Os muitos frameworks de construção de agentes disponíveis no mercado diferem em ergonomia, em vocabulário e em recursos prontos, mas, no essencial, todos instanciam a mesma arquitetura canônica: um reasoning engine que decide, tools que agem, memória que retém e, quando há, planning que decompõe. Ao ler a documentação de qualquer um deles, vale mapear os termos próprios da ferramenta de volta a esses componentes, porque é esse mapeamento que permite comparar opções sem se perder em nomes diferentes para as mesmas coisas. A escolha de framework é uma decisão de produtividade e de integração, não uma escolha sobre o que um agente é.

Com a arquitetura canônica nomeada, falta caracterizar o critério que separa um agente de um sistema que apenas responde, sem agir. Essa distinção é o que organiza a próxima seção e o que, na prática, costuma ser aplicado de forma frouxa, apagando justamente a propriedade que torna a arquitetura agentic mais cara e mais difícil de auditar.

2. Agente e assistente conversacional

Esta seção pertence à camada conceitual e tem por objetivo que você diferencie um agente de um assistente conversacional simples, identificando o que caracteriza a autonomia de um sistema. A distinção importa porque o vocabulário corrente usa a palavra agente de forma frouxa, aplicando-a a qualquer interface baseada em modelo de linguagem. Quando tudo é chamado de agente, a palavra deixa de informar, e decisões de arquitetura passam a ser tomadas sem que ninguém perceba o salto de custo e de imprevisibilidade que elas implicam.

Um assistente conversacional responde a uma entrada e termina. Mesmo quando mantém um histórico de conversa e parece contínuo ao longo de vários turnos, cada resposta é produzida numa única passagem do modelo, sem que o sistema decida agir sobre um ambiente externo e observar consequências antes de responder. O usuário pergunta, o modelo responde, e o controle volta ao usuário. A continuidade que se percebe vem do histórico reapresentado a cada turno, não de um processo autônomo que prossegue sem intervenção.

Um agente, ao contrário, possui agência no sentido técnico: seleciona ações, executa-as por meio de tools, observa os efeitos e itera rumo a um objetivo, sem devolver o controle a cada passo. A propriedade decisiva não é a presença de um modelo de linguagem, comum aos dois, nem a capacidade de chamar uma ferramenta, que um assistente também pode ter de forma pontual. O que distingue o agente é o laço: a existência de um ciclo em que o sistema escolhe e encadeia ações de forma autônoma, decidindo quando continuar e quando parar. Um assistente que consulta o tempo e responde fez uma chamada de ferramenta, mas não iterou; um agente que consulta, avalia o retorno, decide consultar outra coisa e só então responde, esse iterou.

Em vez de classificar uma solução de modo binário, é mais útil situá-la num continuum de autonomia. Num extremo está o chatbot que apenas conversa, sem qualquer ação sobre o mundo. Em seguida vem o assistente que pode chamar uma ferramenta numa única passagem, por exemplo consultar uma cotação e respondê-la, sem laço. Depois, o agente single-loop, que repete o ciclo perceber-decidir-agir com um conjunto controlado de tools e uma condição de parada clara. No extremo de maior autonomia estão os agentes com planejamento de múltiplos passos e, eventualmente, vários agentes coordenados. Cada degrau à direita amplia a capacidade de lidar com tarefas abertas, em

que o caminho não é conhecido de antemão, e cada degrau cobra o preço correspondente em custo de inferência, latência e dificuldade de prever e auditar o comportamento. Situar uma solução nesse continuum, em vez de rotulá-la, é o que permite discutir o que ela ganha e o que ela paga.

Cada degrau do continuum pode ser ilustrado com um caso de PD&I industrial. No degrau do chatbot está um assistente que responde dúvidas sobre uma norma técnica a partir do texto que lhe foi dado, sem consultar nada externo. No degrau seguinte, do assistente com uma chamada de ferramenta, está um sistema que, ao receber a pergunta sobre um equipamento, consulta uma vez a ficha desse equipamento e responde, num único passo. No degrau do agente single-loop está um sistema que, diante de um diagnóstico ambíguo, decide consultar o histórico, avalia o retorno, decide consultar o manual da peça suspeita e só então conclui. No degrau de maior autonomia está um sistema que planeja uma investigação completa, distribui subtarefas e consolida os achados. O mesmo domínio comporta os quatro degraus; o que muda é a abertura do problema que se quer resolver.

O mecanismo que permite a um modelo de linguagem agir, e não apenas escrever, é o tool calling. Em vez de devolver texto livre para o usuário interpretar, o modelo emite uma chamada estruturada que nomeia uma ferramenta e fornece os argumentos esperados, conforme um esquema declarado pelo sistema. O sistema, não o modelo, executa a função correspondente e devolve o resultado, que volta ao modelo como nova observação. Há dois pontos sutis aqui. Primeiro, o modelo propõe a chamada, mas quem a executa é o código ao redor, o que abre o espaço necessário para validar argumentos, recusar chamadas malformadas e tratar erros antes que cheguem ao mundo. Segundo, a estrutura importa: por ser uma chamada com formato declarado, e não texto livre, ela pode ser analisada, validada e registrada de forma confiável.

Entender o tool calling como um contrato estruturado entre modelo e ambiente, e não como uma habilidade mágica do modelo, é o que permite desenhar tools confiáveis. Um contrato tem partes: o nome da ferramenta, a descrição do que ela faz, o formato dos argumentos e o formato do retorno. Quanto mais claro o contrato, menor a chance de o modelo selecionar a ferramenta errada ou preencher os argumentos de forma inadequada. Essa visão também esclarece de onde vêm muitos erros de agentes: não de o modelo querer agir mal, mas de contratos ambíguos, validação ausente e retornos mal formatados que confundem a observação seguinte.

Levar a sério a ideia de contrato tem efeitos concretos na confiabilidade. Como o modelo apenas propõe a chamada e quem a executa é o código ao redor, esse código pode validar se os argumentos respeitam o formato esperado, recusar uma chamada malformada antes que ela produza efeito, e devolver ao modelo uma observação de erro clara

em vez de uma exceção crua. Tools de escrita merecem cuidado adicional, sendo desejável que sejam idempotentes sempre que possível, isto é, que repetir a mesma chamada não cause efeito duplicado, porque um agente em laço pode reintentar uma ação. Esse rigor não decorre de o modelo ser pouco confiável, mas de o contrato ser o ponto em que decisão vira efeito e, portanto, o ponto em que vale concentrar verificação.

GLOSSÁRIO

Tool calling

Mecanismo pelo qual um modelo de linguagem solicita a execução de uma função externa. Em vez de texto livre, o modelo emite uma chamada estruturada, com o nome da ferramenta e os argumentos, segundo um esquema declarado. O sistema executa e devolve o resultado como nova observação. É o contrato que liga decisão a ação.

Quadro 1 - Assistente conversacional e agente, lado a lado.

Aspecto	Assistente conversacional	Agente
Unidade de operação	uma passagem do modelo por resposta	laço de vários passos por objetivo
Ação sobre o ambiente	nenhuma, ou uma chamada pontual	seleção e encadeamento autônomo de tools
Número de passos	fixo (um por resposta)	definido em execução, não de antemão
Custo e latência	previsíveis por resposta	variáveis, crescem com o número de passos
Auditoria	direta, uma entrada e uma saída	exige inspeção do trace de decisões

A confusão entre os dois conceitos não é apenas terminológica; ela tem custo. Tratar como agente o que poderia ser um assistente com uma chamada de ferramenta significa pagar por um laço que não traz benefício. O inverso, tratar como assistente o que exige um agente, significa forçar num único passo uma tarefa que precisa de iteração, produzindo respostas incompletas ou frágeis. O diagnóstico correto começa por reconhecer se a tarefa exige, ou não, encadear ações em função de observações intermedi-

> ⚠ ATENÇÃO
>
> Uma armadilha frequente é supor que a arquitetura agentic é o default moderno, a ser adotado por ser o mais avançado. A consequência é o over-engineering: equipes envolvem em um laço autônomo tarefas cujo caminho é conhecido de antemão, pagando custo, latência e imprevisibilidade sem ganho correspondente. Autonomia é uma escolha condicionada ao problema, não um patamar a ser alcançado.

árias.

A distinção entre agente e assistente não é um preciosismo de vocabulário para o curso; ela organiza o trabalho dos squads. A lente da autonomia atravessa os sete eixos: um squad de engenharia de dados pode considerar um agente para inferência adaptativa de esquema, um de processamento em tempo real pode discutir agentes assíncronos, e um de visão computacional pode avaliar agentes que coordenam ferramentas de percepção. Em todos esses casos, a primeira pergunta útil é a mesma: a tarefa exige encadear ações em função de observações intermediárias, caracterizando um agente, ou cabe numa resposta única, caracterizando um assistente? Partir dessa pergunta, em vez do rótulo da moda, é o que permite uma discussão técnica produtiva entre integrantes de senioridades distintas.

Há ainda uma diferença de estado que convém nomear. Um assistente sem agência pode ser, no limite, sem estado entre requisições: cada resposta depende apenas da entrada e do histórico apresentado. Um agente, por operar em laço, é inerentemente baseado em estado durante a execução, pois acumula observações que condicionam as decisões seguintes, e pode precisar de estado persistente entre execuções, quando retoma uma tarefa de onde parou. Essa presença de estado é parte do que torna o agente mais poderoso e, ao mesmo tempo, mais difícil de testar e reproduzir, já que o mesmo objetivo pode percorrer caminhos diferentes conforme o estado encontrado.

Reconhecida a diferença entre responder e agir, e nomeado o tool calling que a sustenta, o passo seguinte é organizar as formas pelas quais agentes raciocinam ao longo do laço. A próxima seção apresenta esse panorama de paradigmas em alto nível, sem entrar na implementação de cada um.

3. Panorama de paradigmas em alto nível

Esta seção pertence à camada conceitual e tem por objetivo que você reconheça, em

alto nível, os principais paradigmas de raciocínio de agentes e o problema que cada um resolve. O detalhamento de implementação e a comparação instrumentada entre frameworks ficam para o curso de LLMs da Fase 2; aqui interessa o vocabulário que permite ler uma solução e dizer a que família ela pertence, e porque aquela escolha foi feita. Os paradigmas a seguir não são tecnologias rivais que se excluem, mas estratégias que podem inclusive se combinar num mesmo sistema.

O paradigma mais difundido é o ReAct, contração de reasoning and acting. Nele, o agente intercala passos de raciocínio explícito e passos de ação num mesmo laço: registra um raciocínio sobre o que precisa, escolhe uma ação, observa o resultado, raciocina de novo à luz da observação e prossegue. Em prosa, um trace de ReAct se lê como uma alternância de pensamento, ação e observação: o agente pensa que precisa de um dado, age consultando uma fonte, observa o valor retornado, pensa que ainda falta outro dado, age de novo, e assim por diante até concluir. Essa alternância traz dois ganhos. O raciocínio intercalado torna o comportamento mais transparente, porque cada ação vem acompanhada de uma justificativa textual que pode ser inspecionada; e torna o sistema mais robusto, porque o agente reavalia o curso a cada observação, em vez de seguir um plano cego. O custo é o número de chamadas ao modelo, que cresce com o número de passos do laço.

GLOSSÁRIO

ReAct

Paradigma de agente que intercala passos de raciocínio e passos de ação num mesmo laço. A cada iteração, o agente registra um raciocínio, escolhe uma ação, executa-a e incorpora a observação resultante antes do raciocínio seguinte. Tende a ser transparente e adaptável, ao custo de mais chamadas ao modelo.

Vale tornar o ReAct concreto com um trace em prosa. Suponha o objetivo de informar se um lote de produção ficou dentro da tolerância especificada. O agente raciocina que precisa primeiro da medição do lote, age consultando a tool de medições, observa o valor retornado; raciocina que precisa agora da especificação de tolerância, age consultando a tool de especificações, observa o intervalo permitido; raciocina que pode comparar os dois, age chamando a tool de cálculo, observa que o valor está dentro do intervalo; e, julgando o objetivo atingido, encerra com a resposta. Cada ação foi precedida de um raciocínio inspecionável e decidida à luz da observação anterior. Se a medição tivesse vindo ausente, o mesmo agente teria raciocinado por uma fonte alternativa, sem que esse desvio precisasse estar previsto num plano fixo.

Um segundo paradigma é o plan-and-execute, que separa em duas fases o que o ReAct entrelaça. Primeiro o agente elabora um plano, uma sequência de subtarefas que levam ao objetivo; depois executa cada passo do plano, eventualmente revisando-o se as observações divergirem do esperado. A separação favorece tarefas longas e estruturadas, em que pensar o todo antes de começar evita desvios e retrabalho, e em que um plano explícito também facilita a supervisão humana, pois pode ser revisado antes da execução. A fragilidade aparece quando o ambiente muda durante a execução e o plano inicial perde validade; daí a importância de o paradigma prever a revisão do plano, e não apenas a sua execução cega.

O terceiro paradigma é a reflexão. Em vez de aceitar a primeira saída, o agente avalia criticamente o próprio resultado e tenta melhorá-lo numa nova passagem, usando a autocrítica como insumo. A reflexão eleva a qualidade sobretudo em tarefas com critério de sucesso verificável, como gerar código que precisa passar em testes ou produzir um texto que deve satisfazer uma lista de requisitos: nesses casos, a verificação fornece um sinal objetivo que orienta a próxima tentativa. Onde não há critério verificável, a reflexão corre o risco de apenas reescrever a saída de forma mais convincente, sem necessariamente torná-la mais correta, e a cada ciclo adicional acrescenta chamadas ao modelo.

O quarto paradigma é o multi-agente, em que vários agentes especializados colaboram, cada um com um papel, uma instrução e um conjunto de tools próprios, coordenados por um agente orquestrador ou por um protocolo de troca de mensagens. O arranjo ajuda quando a tarefa se decompõe naturalmente em papéis distintos, por exemplo um agente que pesquisa, outro que redige e um terceiro que revisa, e quando manter cada papel com instruções enxutas melhora a qualidade frente a um único agente sobrecarregado. O preço é alto: o custo se multiplica com o número de agentes, e surgem modos de falha de coordenação, como agentes que repetem trabalho, divergem sobre o objetivo ou propagam o erro de um para o outro. Por isso o curso trata o multi-agente como recurso a ser justificado, e não como ponto de partida.

Os modos de falha de coordenação do paradigma multi-agente merecem ser nomeados, porque são contraintuitivos para quem espera que mais agentes signifiquem sempre mais capacidade. Agentes podem repetir trabalho quando seus papéis se sobrepõem sem fronteira clara; podem divergir sobre o objetivo quando a instrução de cada um o interpreta de modo distinto; e podem propagar erro, quando a saída equivocada de um vira entrada confiável do seguinte, amplificando o engano em vez de corrigi-lo. Esses modos não invalidam o paradigma, mas explicam por que a coordenação, e não a quantidade de agentes, é o que determina se o arranjo ajuda. Em PD&I industrial, começar com um único agente bem instruído e só decompor em papéis quando a sobrecarga de um deles ficar evidente costuma ser o caminho mais seguro.

Quadro 2 - Quatro paradigmas de raciocínio de agentes em alto nível.

Paradigma	Ideia central	Quando tende a ajudar
ReAct	intercala raciocínio e ação no mesmo laço	tarefas exploratórias com correção de curso
Plan-and-execute	planeja antes, executa e revisa depois	tarefas longas e estruturadas
Reflexão	autocrítica e nova tentativa	tarefas com critério de sucesso verificável
Multi-agente	papéis especializados que colaboram	tarefas decomponíveis em papéis distintos

Convém também nomear quando cada paradigma tende a atrapalhar, porque reconhecer o mau encaixe é tão útil quanto reconhecer o bom. O ReAct, por intercalar uma chamada de modelo a cada passo, fica caro e lento quando a tarefa tem muitos passos triviais que não precisariam de raciocínio; nesses casos, um workflow resolve melhor. O plan-and-execute sofre quando o ambiente é imprevisível e o plano inicial envelhece rápido, situação em que a adaptação contínua do ReAct é preferível. A reflexão desperdiça chamadas quando não há um critério objetivo que diga se a nova tentativa é de fato melhor. O multi-agente penaliza tarefas simples, em que a coordenação entre agentes custa mais do que o ganho de especialização. A pergunta correta nunca é qual o paradigma mais avançado, mas qual a dificuldade específica do problema e qual estratégia a enfrenta com o menor custo.

Dois alertas fecham o panorama. O primeiro é que esses paradigmas se combinam: é comum um sistema multi-agente em que cada agente raciocina no estilo ReAct, ou um plano de alto nível cujos passos individuais são executados com reflexão. O segundo é que sofisticação não é sinônimo de adequação. Adicionar reflexão, planejamento ou múltiplos agentes só se justifica quando o problema apresenta a dificuldade que aquele recurso resolve; do contrário, acrescenta-se custo e fragilidade sem ganho.

SAIBA MAIS

O paradigma ReAct é apresentado em YAO, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models (ICLR, 2023, disponível em https://arxiv.org/abs/2210.03629). A comparação instrumentada entre paradigmas e frameworks é aprofundada no curso de LLMs da Fase 2 do programa.

Esses paradigmas descrevem como um agente raciocina, mas deixam em aberto como ele se conecta de forma padronizada às ferramentas e fontes de dados de que depende.

4. Model Context Protocol como padrão de integração

Esta seção pertence à camada conceitual e tem por objetivo que você reconheça o Model Context Protocol como um padrão emergente de integração entre agentes e o mundo externo, e entenda o problema que ele endereça. O objetivo é situar o padrão e o que ele sinaliza, não o implementar; o detalhamento operacional cabe ao curso de LLMs da Fase 2.

À medida que agentes passam a depender de muitas ferramentas e fontes de dados, surge um problema de combinatória que vale a pena tornar explícito. Sem um padrão comum, cada agente precisa de uma integração feita sob medida para cada ferramenta, e cada ferramenta precisa ser adaptada a cada agente que a use. Com dois ou três sistemas isso é tolerável; com dezenas, o número de integrações a construir e manter cresce com o produto entre a quantidade de agentes e a de ferramentas. O resultado é um emaranhado de conectores específicos, caro de manter e frágil a mudanças: alterar uma ferramenta obriga a rever todas as integrações que a consomem.

O Model Context Protocol, ou MCP, é uma proposta de protocolo aberto para reduzir esse acoplamento. Ele define uma interface uniforme, no estilo cliente e servidor, pela qual um agente descobre e invoca ferramentas e acessa fontes de dados expostas por servidores que falam o mesmo protocolo. O efeito prático é transformar um problema de muitas integrações ponto a ponto num problema em que cada lado fala o protocolo uma única vez: quem constrói uma ferramenta a expõe como servidor compatível, e qualquer agente que seja cliente do protocolo passa a poder usá-la sem integração específica. Em vez de o número de conectores crescer com o produto entre agentes e ferramentas, ele passa a crescer com a soma de cada lado, o que muda a escala do problema de manutenção.

GLOSSÁRIO

Model Context Protocol (MCP)

Protocolo aberto que padroniza a conexão entre agentes e ferramentas ou fontes de dados, no modelo cliente e servidor. Um agente compatível descobre e invoca recursos expostos por servidores que falam o protocolo, sem integração sob medida para cada par. Reduz o acoplamento entre o desenvolvimento de agentes e o de ferramentas.

Para tornar a combinatória concreta, imagine cinco agentes que precisam acessar oito ferramentas. Sem padrão comum, há até quarenta integrações possíveis a construir e manter, e cada nova ferramenta acrescenta cinco integrações, uma por agente. Com um protocolo comum, cada agente passa a falar o protocolo uma vez, e cada ferramenta o expõe uma vez, de modo que treze adaptações substituem as quarenta, e uma nova ferramenta acrescenta apenas uma. Os números exatos importam menos do que a mudança de regime: deixa-se de pagar pelo produto entre os dois lados para pagar pela soma deles, e é essa mudança que torna sustentável conectar muitos agentes a muitas fontes.

Por trás da interface uniforme há dois movimentos que vale nomear. O primeiro é a descoberta: um agente cliente consulta um servidor compatível e recebe a lista de recursos que ele oferece, com a descrição de cada um e o formato esperado de uso, de modo que o agente passa a saber o que pode invocar sem que isso tenha sido programado caso a caso. O segundo é a invocação padronizada: uma vez descoberto um recurso, o agente o aciona pelo mesmo mecanismo, qualquer que seja a ferramenta por trás. É a combinação de descoberta e invocação uniformes que reduz a integração a configurar um cliente em vez de escrever código de cola.

Sob a interface uniforme, um servidor compatível costuma expor mais de um tipo de recurso. Há recursos de contexto, que fornecem dados para fundamentar as respostas do agente, e há recursos de ação, as ferramentas propriamente ditas, que executam operações. (Na especificação do protocolo, essas categorias funcionais correspondem, em termos próprios, às primitivas Resources, Prompts e Tools, cujo detalhamento cabe ao curso de LLMs da Fase 2.) Essa separação ecoa uma distinção que o curso já encontrou ao tratar de geração aumentada por recuperação, no módulo anterior: trazer contexto relevante para a janela do modelo é um problema, e agir sobre o mundo é outro. Um padrão de integração que cobre os dois sob a mesma interface reduz o atrito de montar agentes que precisam, ao mesmo tempo, consultar conhecimento atualizado e executar ações sobre sistemas.

O movimento que o MCP representa não é inédito na história da computação. Sempre que muitos produtores e muitos consumidores precisaram se conectar, padrões de interface reduziram o custo de integração ao substituir acordos par a par por uma convenção comum, como ocorreu com formatos de documento, protocolos de rede e interfaces de dispositivo. A novidade aqui é o objeto padronizado: não dados nem páginas, mas o acesso de um agente a capacidades de ação e a fontes de contexto. Reconhecer esse paralelo ajuda a calibrar a expectativa, pois padrões assim levam tempo para amadurecer e conviver com alternativas antes de se estabilizarem.

Conectar um agente a ferramentas por um protocolo também levanta uma questão de confiança que não se deve ignorar. Um servidor descreve o que oferece, e um agente que confia cegamente nessa descrição pode ser induzido a ações indevidas por um servidor mal-intencionado ou comprometido. Em ambiente industrial, isso recomenda tratar servidores como fronteiras de confiança: usar apenas os homologados, validar o que retornam e limitar o que cada um pode acionar. O padrão reduz o custo de integração, mas não dispensa o cuidado com a procedência das ferramentas conectadas.

> **Exemplo aplicado**
>
> Uma equipe de engenharia de dados mantém um servidor que expõe consultas a um catálogo de dados interno. Ao publicá-lo segundo um protocolo de integração padronizado, diferentes agentes do programa, de squads distintos, passam a consultar o catálogo sem que cada squad escreva a própria integração. O custo de conectar um novo agente ao catálogo cai de uma integração feita sob medida para uma simples configuração de cliente.

A decisão de construir uma integração própria ou consumir uma já disponível, que o padrão torna mais frequente, tem contornos reconhecíveis em PD&I industrial. Consumir um servidor já homologado economiza esforço e tende a ser preferível quando a ferramenta é genérica e bem mantida. Construir a própria integração se justifica quando a ferramenta é específica do domínio, quando há requisitos de segurança que um servidor genérico não atende, ou quando o controle sobre o comportamento da integração é parte do valor da solução. O padrão não resolve essa decisão, mas a torna mais limpa, ao separar o que é integração reutilizável do que é lógica específica do projeto.

A relevância do MCP para este curso é menos a sua especificação técnica e mais o que ele sinaliza para quem decide arquiteturas em PD&I industrial: a integração de agentes com sistemas existentes está deixando de ser um esforço artesanal por solução para se tornar uma camada padronizada. Isso muda a conta de custo de adoção, porque ferramentas já expostas via protocolo podem ser reaproveitadas entre projetos, e influencia uma decisão recorrente, a de construir uma integração própria ou consumir uma já disponível. Convém, ainda assim, tratar o padrão com a cautela devida a uma tecnologia emergente, cujo ecossistema ainda está em formação.

Com os componentes, a distinção em relação ao assistente, os paradigmas de raciocínio e o padrão de integração estabelecidos, encerra-se a camada conceitual do módulo. A camada aplicada, que começa na próxima seção, desce do vocabulário para o reconhecimento de padrões concretos em especificações de PD&I industrial.

5. Dois padrões aplicados em PD&I industrial

Esta seção pertence à camada aplicada e tem por objetivo que você reconheça os dois padrões que aparecem com maior frequência em projetos de PD&I industrial e identifique os componentes mínimos de cada um. A escolha justificada entre eles, com seus eixos de comparação, é tratada na camada decisória, no próximo e-book; aqui o foco é caracterizar os padrões com precisão suficiente para identificá-los numa especificação.

O primeiro padrão é o agente single-loop com tools controladas. Há um único reasoning engine que repete o ciclo perceber-decidir-agir sobre um conjunto pequeno e curado de ferramentas, com uma condição de parada explícita. O termo single-loop indica o que esse padrão não tem: não há hierarquia de planos elaborados de antemão, não há múltiplos agentes coordenados, não há orquestração complexa. Há um laço, poucas tools bem definidas e limites claros, como um número máximo de iterações. A simplicidade é deliberada, porque mantém o comportamento mais previsível e o trace mais fácil de inspecionar. É o padrão indicado quando a tarefa exige decidir qual ação tomar a seguir em função do que se observa, mas o espaço de ações é restrito e auditável.

Convém detalhar o que mantém o agente single-loop sob controle. Três limites costumam acompanhá-lo. O primeiro é a lista restrita de tools, escolhida de modo que cada ação possível seja conhecida e auditável. O segundo é o teto de iterações, que impede o laço de prosseguir indefinidamente e funciona como salvaguarda de custo. O terceiro é a condição de parada bem definida, que diz ao agente quando o objetivo foi atingido. Esses limites não são opcionais: são o que separa um agente single-loop confiável de um laço que pode consumir recursos sem convergir. Mantê-los explícitos desde o desenho é parte de aplicar bem o padrão.

Para firmar o reconhecimento do agente single-loop, considere dois casos opostos. Encaixa-se bem o atendimento de uma solicitação de diagnóstico em que, conforme os sintomas relatados, o sistema decide quais verificações rodar e em que ordem, parando quando a causa fica identificada; o caminho varia por solicitação e exige decisão a cada passo. Não se encaixa o processamento de um formulário com campos sempre iguais, validação fixa e destino determinado pelo conteúdo de um campo conhecido; aqui não há decisão de caminho a tomar, e envolver isso num laço autônomo só adiciona custo e imprevisibilidade. Contrastar um caso de bom encaixe com um de mau encaixe treina o olhar para a propriedade que de fato importa, que é a variabilidade do caminho.

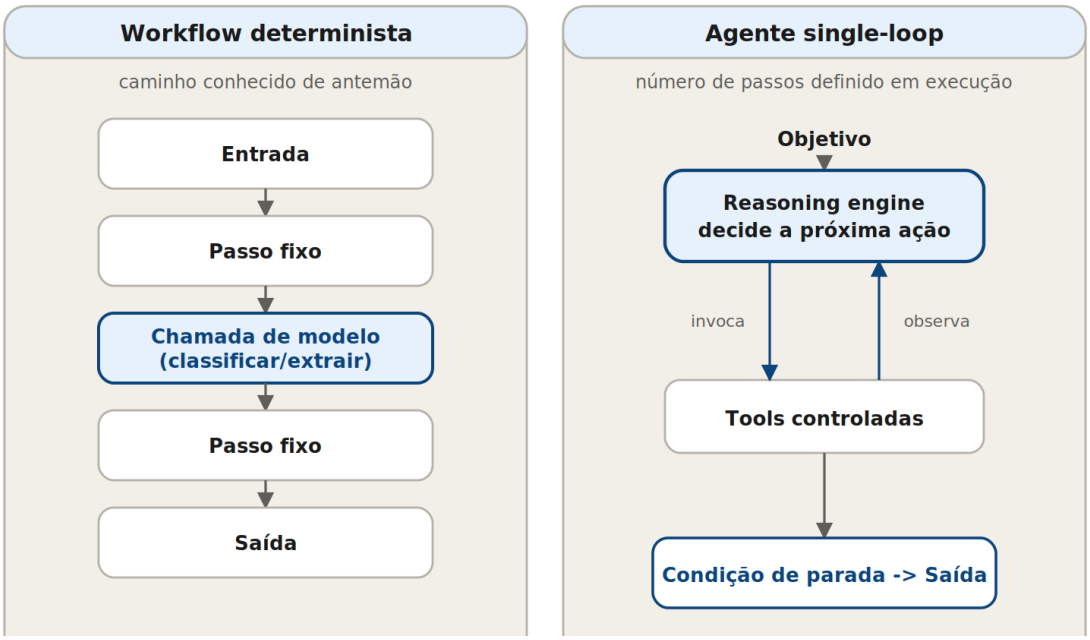
O segundo padrão é o workflow determinista com chamadas pontuais de modelo. Aqui a estrutura de controle é fixa: um grafo de passos definido por quem projetou o sistema, em que o modelo é chamado em pontos específicos para executar uma subtarefa bem delimitada, como classificar um documento, extrair campos de um texto ou gerar um resumo. A diferença essencial em relação ao agente é que, no workflow, o modelo não

decide o fluxo; ele preenche lacunas dentro de um fluxo que já está decidido. As ramificações que existam são condicionais programadas, do tipo se o tipo é A siga por aqui, e não escolhas que o modelo faz livremente a cada passo. É o padrão indicado quando os passos são conhecidos de antemão e a variação relevante está no conteúdo processado, não no caminho percorrido.

Do lado do workflow, percorrer um pipeline concreto ajuda a fixar o padrão. Num fluxo de processamento de documentos, a entrada é recebida, um nó chama o modelo para classificar o tipo de documento, uma ramificação programada encaminha cada tipo para o extrator adequado, outro nó chama o modelo para extrair os campos previstos, um nó de validação verifica os dados contra regras, e a saída é gravada. O modelo foi acionado em dois pontos bem delimitados, sempre os mesmos, e em nenhum momento decidiu o fluxo; quem decidiu o fluxo foi quem desenhou o grafo. O número de chamadas ao modelo é conhecido, e por isso custo e latência são estimáveis antes de qualquer execução.

Também vale enumerar as subtarefas para as quais o modelo costuma ser chamado em pontos fixos do grafo, porque é nesses pontos que a capacidade do modelo entra sem que ele assuma o controle do fluxo. As mais comuns em PD&I industrial são classificar uma entrada numa categoria conhecida, extrair campos estruturados de um texto livre, rotear um item para a fila correta segundo seu conteúdo, resumir um documento e gerar um texto a partir de dados estruturados. Em todos esses casos a tarefa do modelo é local e bem delimitada, e o que vem antes e depois dela está fixado pelo projetista. O workflow combina, assim, a previsibilidade de um fluxo programado com a flexibilidade do modelo nos pontos em que ela é necessária.

Figura 2 - Workflow determinista e agente single-loop, lado a lado.



Fonte: autor.

Figura 2 intitulada "Workflow determinista versus agente single-loop", organizada em duas colunas lado a lado. A coluna da esquerda, denominada "Workflow determinista", representa um processo linear e previamente definido, identificado pela frase "caminho conhecido de antemão". O fluxo é composto por cinco blocos retangulares conectados por setas apontando para baixo: Entrada, Passo fixo, Chamada de modelo (classificar/extrair), Passo fixo e Saída, ilustrando uma sequência rígida de execução, sem desvios ou tomada de decisão.
Na coluna da direita, intitulada "Agente single-loop", aparece a descrição "número de passos definido em execução", indicando um processo mais adaptativo. No topo está o bloco Objetivo, seguido do bloco central "Reasoning engine decide a próxima ação", destacado em azul. A partir dele, uma seta desce para o bloco "Tools controladas", indicando a execução de uma ação. Outra seta retorna das ferramentas para o mecanismo de decisão, formando um ciclo contínuo de observação e escolha da próxima ação, com as palavras "invoca" e "observa" indicando esse fluxo bidirecional. Na parte inferior há o bloco "Condição de parada Saída", mostrando que o processo se repete até que o objetivo seja alcançado ou uma condição de encerramento seja satisfeita. A figura evidencia o contraste entre um fluxo fixo e previsível e um agente capaz de decidir dinamicamente seus próximos passos durante a execução. Na parte inferior consta a legenda "Fonte: autor."

A diferença estrutural entre os dois padrões reduz-se a uma pergunta: quem decide o caminho? No workflow, o caminho é decidido pelo projetista e fixado no código, e o modelo opera dentro dele. No agente, o caminho é decidido pelo reasoning engine em tempo de execução, em função das observações. Essa diferença tem consequência direta de custo e previsibilidade. O workflow faz um número conhecido de chamadas ao modelo, o que torna custo e latência estimáveis com precisão; o agente pode fazer muitas chamadas, e cada iteração do laço acrescenta uma chamada e, com ela, latência e custo de inferência. O workflow é mais fácil de auditar, porque o caminho é sempre o mesmo; o agente exige inspecionar o trace de decisões de cada execução, que difere de caso para caso.

A consequência de custo merece um contraste numérico em ordem de grandeza. Um workflow que chama o modelo duas vezes por requisição tem custo e latência essencialmente constantes, fáceis de multiplicar pelo volume esperado. Um agente que, conforme o caso, faz de três a quinze iterações tem custo que varia na mesma proporção, e o planejamento de capacidade precisa considerar não a média, mas também o pior caso, sob pena de subdimensionar a infraestrutura e estourar o orçamento em picos. Essa assimetria de previsibilidade, e não apenas o custo médio, é parte do que se pondera ao escolher entre os padrões.

Os dois padrões são pontos de referência, não uma dicotomia rígida. Existem híbridos úteis, como um workflow determinista que, em um único nó, aciona um pequeno agente single-loop para uma subtarefa aberta, e depois retoma o fluxo fixo. Reconhecer os dois padrões puros é o que permite, em seguida, descrever esses híbridos com clareza, dizendo onde o fluxo é fixo e onde ele é decidido em execução.

Exemplo aplicado

Numa seguradora, a triagem de documentos de sinistro segue passos sempre iguais: classificar o tipo de documento, extrair os campos previstos e encaminhar para a fila correta. Um workflow determinista chama o modelo duas vezes, para classificar e para extrair, dentro de um fluxo fixo. Já a investigação de um sinistro suspeito, que pode exigir consultar bases diferentes em ordem variável conforme o que se encontra, encaixa-se melhor num agente single-loop com tools controladas.

⚠ ATENÇÃO

Armadilha comum

Um erro recorrente é subestimar como custo e latência crescem em arquiteturas agênticas. Como o agente repete o laço e cada iteração é uma nova chamada ao modelo, o custo não cresce de forma linear e previsível como num workflow, mas de forma multiplicativa com o número de passos, que varia por execução. Tratar um agente como se tivesse o perfil de custo de uma única chamada produz orçamentos e estimativas de latência irreais.

Caracterizados os dois padrões e a diferença de quem decide o caminho, falta a habilidade prática de olhar para uma especificação e dizer qual padrão está em jogo.

6. Reconhecimento do padrão em uma especificação

Esta seção pertence à camada aplicada e tem por objetivo que você reconheça, a partir da especificação de um problema, qual dos dois padrões tende a ser adequado. O reconhecimento aqui é descritivo: trata-se de identificar o padrão sugerido pelas características do problema. A decisão justificada, com seus eixos de comparação e a heurística de quando subir de workflow para agente, pertence à camada decisória e é trabalhada no próximo e-book. Reconhecer não é ainda decidir, mas é o reconhecimento que torna a decisão possível.

Para classificar uma especificação na prática, vale aplicar uma sequência curta de perguntas. A primeira é se o conjunto de passos necessários é conhecido de antemão. Se o caminho da entrada à saída pode ser desenhado como um grafo fixo antes de qualquer execução, o problema tende ao workflow determinista. Se o próximo passo depende do que se descobrir no passo anterior, de um modo que não dá para enumerar antecipadamente, há indício de agente. O teste mental é direto: você consegue desenhar o fluxograma completo sem saber qual será a entrada específica? Se sim, é provável que um workflow baste.

A segunda pergunta é como se decide a ramificação do fluxo. Bifurcações previsíveis, decidíveis por regras simples sobre dados estruturados, do tipo se a categoria é A siga por aqui, cabem num workflow com condicionais. Quando a escolha do próximo passo exige interpretar conteúdo aberto e variável, e essa interpretação não se reduz a um conjunto fechado de regras, o reasoning engine de um agente passa a se justificar. A terceira pergunta é quantas chamadas ao modelo a tarefa exige por requisição e se esse número é fixo ou variável. Número fixo e pequeno aponta para workflow; número

variável, dependente do caso concreto, aponta para agente. A quarta pergunta é qual o orçamento de custo e latência e qual o nível de auditabilidade exigido. Restrições apertadas de custo, latência previsível ou auditoria rigorosa favorecem o workflow, cujo comportamento é mais fácil de prever e inspecionar; folga nesses requisitos torna o agente viável.

Vale percorrer a sequência num exemplo curto. Uma especificação pede um sistema que receba uma nota fiscal em PDF, extraia os itens, valide cada item contra um cadastro e gere um relatório de divergências. Os passos são conhecidos, a ramificação se resolve por regras de validação, o número de chamadas ao modelo é fixo, e há exigência de auditoria. As quatro respostas convergem para workflow determinista. Mude a especificação para um sistema que investigue a causa de uma divergência recorrente, consultando bases diferentes conforme as pistas que encontra, e as respostas se invertem: passos não conhecidos de antemão, ramificação que exige interpretação, número variável de consultas. O mesmo domínio, notas fiscais, abriga problemas que pedem padrões diferentes.

Quadro 3 - Sequência de reconhecimento mapeada aos dois padrões.

Pergunta sobre a especificação	Indica workflow	Indica agente
Os passos são conhecidos de antemão?	Sim, grafo fixo	Não, dependem do que se descobre
Como se decide a ramificação?	Regras e condicionais simples	Interpretação de conteúdo aberto
Quantas chamadas ao modelo por requisição?	Número fixo e pequeno	Número variável por caso
Orçamento de custo, latência e auditoria?	Apertado ou rigoroso	Folgado e tolerante a variação

Há um princípio que organiza a leitura dessas perguntas e antecipa a heurística decisória do próximo e-book: comece pela solução mais simples que resolve o problema e só adote a complexidade que o problema exigir. Na prática, isso significa preferir o workflow sempre que ele baste, e reservar o agente para os casos em que a abertura do problema o torna necessário. A sequência de reconhecimento serve justamente a esse princípio: ela não busca encontrar uma razão para usar um agente, mas verificar se o problema realmente apresenta a imprevisibilidade de caminho que só um agente resolve bem.

Um terceiro exemplo, propositalmente intermediário, mostra como a sequência lida com casos que não são puros. Uma especificação pede um sistema que processe pedidos de compra: na maior parte do tempo, os passos são fixos, validar o pedido, conferir o orçamento, registrar e notificar, o que aponta claramente para workflow. Mas há uma etapa em que, diante de uma exceção de orçamento, o sistema deve investigar alternativas consultando fornecedores e histórico em ordem que depende do que encontra. As três primeiras perguntas apontam para workflow no fluxo principal, e a etapa de exceção aponta para agente. O reconhecimento correto não é escolher um dos dois para tudo, mas identificar que o fluxo principal é um workflow que, num nó específico, aciona um pequeno agente para a subtarefa aberta. Localizar a autonomia onde ela é necessária, em vez de espalhá-la, é o ganho da leitura cuidadosa.

Reconhecer o padrão também antecipa onde estará o risco principal de cada escolha. Num workflow, o risco concentra-se na qualidade das chamadas pontuais ao modelo, por exemplo uma classificação errada que segue fluxo adiante sem correção, e na rigidez do fluxo diante de casos não previstos. Num agente, o risco concentra-se no laço: seleção de tool equivocada, iterações que não convergem, custo que escapa do previsto. Saber, desde a leitura da especificação, qual família de risco está em jogo permite ao squad planejar as mitigações certas em vez de descobri-las em produção.

Esse reconhecimento alimenta diretamente o trabalho dos squads. Ao delimitar um problema técnico, descrever uma solução proposta como workflow, como agente ou como híbrido, e justificar essa leitura com as quatro perguntas, traduz uma intenção vaga em características técnicas comparáveis, que cabem no Mapa Técnico do Problema. Mais do que rotular, o que se ganha é uma linguagem comum: integrantes de formações distintas passam a discutir a mesma especificação nos mesmos termos, e o desacordo, quando há, recai sobre uma pergunta concreta, como se o número de chamadas ao modelo é fixo, e não sobre impressões gerais.

A combinação das respostas posiciona a especificação mais próxima de um padrão ou de outro e, tão importante quanto isso, expõe onde estarão os principais riscos de execução. Quando as respostas divergem, apontando parte para workflow e parte para agente, o sinal costuma ser de um híbrido, em que a maior parte do fluxo é fixa e um trecho específico exige decisão em execução. Esse caso intermediário não enfraquece a sequência de reconhecimento; ao contrário, é ela que permite localizar com precisão o trecho que demanda autonomia, em vez de agentificar o sistema inteiro por causa de uma única etapa aberta.

SAIBA MAIS

Para Aprofundar

A orientação de começar pela solução mais simples e só subir para um agente quando o problema o justifica é argumentada em ANTHROPIC. Building Effective Agents (Anthropic Engineering, 2024, disponível em https://www.anthropic.com/engineering/building-effective-agents). A formalização dessa escolha como heurística decisória é o tema do próximo e-book.

7. Conclusão

Estas seis seções cobriram, em conjunto, as camadas conceitual e aplicada da lente da autonomia. Na camada conceitual, fixaram a arquitetura canônica de um agente, com reasoning engine, tools, memória e planning articulados no ciclo perceber-decidir-agir; a distinção entre agente e assistente conversacional, ancorada na presença de um laço autônomo de ação e no mecanismo de tool calling que o sustenta; o panorama de paradigmas de raciocínio em alto nível, com ReAct, plan-and-execute, reflexão e multi-agente; e o Model Context Protocol como padrão emergente de integração. Na camada aplicada, caracterizaram os dois padrões mais frequentes em PD&I industrial, o agente single-loop com tools controladas e o workflow determinista com chamadas pontuais de modelo, e ofereceram uma sequência de perguntas para reconhecer qual padrão uma especificação sugere.

O fio que une as seis seções é a diferença entre decidir o caminho de antemão e decidir o caminho em execução. Essa diferença, mais do que qualquer rótulo de framework, é o que separa um workflow de um agente, e é também o que governa custo, latência e auditabilidade. Nomeá-la com precisão é o ganho central deste texto-base: permite que pessoas de senioridades e formações distintas, num mesmo squad, descrevam uma solução proposta usando os mesmos termos e localizem, desde a leitura da especificação, onde estarão os riscos de execução.

Vale reter três armadilhas que o texto nomeou, porque elas reaparecem na prática industrial. A primeira é tratar a arquitetura agentic como default moderno, adotando autonomia por sofisticação e não por necessidade. A segunda é entender o tool calling como uma habilidade do modelo, e não como um contrato estruturado que precisa de validação e tratamento de erro. A terceira é estimar o custo de um agente como se fosse o de uma única chamada, ignorando que o laço multiplica chamadas em número variável. As três têm a mesma raiz: confundir o que o sistema pode fazer com o que o problema exige.

Este texto-base deliberadamente parou antes da decisão. Reconhecer que uma especificação sugere um agente não é o mesmo que decidir construí-lo, porque a decisão pondera o ganho real da autonomia contra seu custo e sua opacidade. Essa ponderação, organizada como a heurística workflow versus agente e acompanhada da análise de modos de falha e mitigações, é o conteúdo da camada decisória do módulo, tratada no próximo e-book, e é exercitada na prática no notebook single-loop do notebook Colab (a seguir), em que o trace de raciocínio de um agente é inspecionado passo a passo.

Ao final do módulo, o repertório construído aqui sustenta diretamente o encontro presencial em que a solução é desenhada, momento em que a opção por um agente passa a exigir registro explícito do custo de inferência e da estratégia de observabilidade. A lente da autonomia, como as demais lentes do curso, não é uma tecnologia paralela a ser dominada, mas um ângulo pelo qual qualquer problema de IA do programa pode ser lido com mais discernimento.

REFERÊNCIAS

Bibliografia mínima

YAO, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023. Preprint em https://arxiv.org/abs/2210.03629.

SCHICK, T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023. Preprint em https://arxiv.org/abs/2302.04761.

ANTHROPIC. Building Effective Agents. Anthropic Engineering, 2024. Disponível em https://www.anthropic.com/engineering/building-effective-agents.

ANTHROPIC. Model Context Protocol Specification. modelcontextprotocol.io, edição em curso. Disponível em https://modelcontextprotocol.io.

Bibliografia complementar

WANG, L. et al. A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 2024. Preprint em https://arxiv.org/abs/2308.11432.

PARK, J. S. et al. Generative Agents: Interactive Simulacra of Human Behavior. UIST, 2023. Preprint em https://arxiv.org/abs/2304.03442.

SHINN, N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023. Preprint em https://arxiv.org/abs/2303.11366.

LANGCHAIN. LangGraph Documentation. langchain-ai.github.io/langgraph, edição em curso. Disponível em https://langchain-ai.github.io/langgraph.

WU, Q. et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023. Preprint em https://arxiv.org/abs/2308.08155.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.